GharSeva

AI-Powered Home Services Marketplace

SOFTWARE PROJECT DOCUMENTATION

Master of Computer Applications (MCA)

Technology Stack

React.js + Tailwind CSS | FastAPI (Python) | PostgreSQL | JWT | SQLAlchemy

Document Version: 1.0
Date: July 1, 2026
Status: Final

2. Table of Contents

3. Executive Summary

GharSeva is an AI-powered home services marketplace designed to bridge the gap between homeowners seeking reliable service professionals and skilled workers looking for consistent employment opportunities. The platform leverages modern artificial intelligence and machine learning techniques to intelligently match service requests with qualified providers, automate scheduling, predict demand, and ensure quality outcomes through data-driven feedback loops.

The home services industry in India is valued at over USD 5 billion and is growing at a CAGR of approximately 20%. Despite significant demand, the sector remains highly fragmented, unorganized, and prone to trust and quality issues. GharSeva addresses these pain points by offering a transparent, technology-driven marketplace that empowers customers and professionals alike.

3.1 Key Highlights

Dimension	Detail
Platform Type	Web-based Marketplace (Mobile-Responsive)
Core AI Feature	Smart service provider matching using preference scoring
Target Audience	Urban homeowners and certified home service professionals
Primary Geography	India (Tier-1 and Tier-2 cities)
Monetization	Commission per booking + Premium provider listings
Compliance	GDPR-aligned data practices, JWT-secured APIs

The system is built using a modern, cloud-native technology stack including React.js with Tailwind CSS for the frontend, FastAPI for a high-performance Python backend, PostgreSQL for relational data management, SQLAlchemy as the ORM, and JWT for secure stateless authentication. The architecture is designed for horizontal scalability and cloud deployment on AWS or GCP.

4. Business Requirement Document (BRD)

4.1 Business Background

India's home services market is dominated by unorganized local vendors, word-of-mouth referrals, and informal labor markets. Customers face challenges in finding vetted, reliable professionals, while skilled workers struggle with inconsistent work pipelines. Digital aggregators have begun addressing this gap, but few leverage AI for intelligent matching and quality control.

4.2 Business Objectives

- Create a trusted digital marketplace connecting homeowners with verified service professionals.
- Reduce booking friction by enabling customers to book services in under three minutes.
- Leverage AI to match customers with the most suitable provider based on availability, ratings, location, and specialty.
- Provide service professionals with a consistent, reliable workflow and tools for self-management.
- Generate revenue through service commissions and optional premium memberships for professionals.
- Establish a quality assurance loop via post-service ratings, reviews, and dispute resolution.

4.3 Business Scope

The GharSeva platform will cover the following service verticals in its initial release:

Service Category	Examples	Complexity
Cleaning	Home cleaning, deep cleaning, sofa/carpet cleaning	Low
Plumbing	Leak repair, pipe fitting, drain unclogging	Medium
Electrical	Wiring, appliance repair, switchboard installation	Medium-High
Carpentry	Furniture repair, custom woodwork, door/window fixes	Medium
Painting	Interior/exterior painting, wall texturing	Medium
Pest Control	General pest control, termite treatment	Medium
Appliance Repair	AC, refrigerator, washing machine servicing	High
Gardening	Lawn mowing, pruning, plant care	Low

4.4 Business Assumptions

- Internet penetration in target cities is adequate for platform adoption.
- Payment gateways (Razorpay/Stripe) are available for digital transactions.
- Service professionals can be onboarded with ID, skill, and background verification.
- The MVP will be launched in three metro cities before a pan-India rollout.

4.5 Business Constraints

- Budget and timeline are limited to MCA project scope for the MVP phase.
- Third-party background verification integration is dependent on external API availability.

- Regulatory compliance with local labor laws for gig workers must be observed.

4.6 Success Criteria

KPI	Target (6 Months Post-Launch)
Monthly Active Users (MAU)	10,000+
Service Bookings/Month	5,000+
Provider Acceptance Rate	> 80%
Customer Satisfaction Score	> 4.2 / 5.0
Booking Completion Rate	> 90%
Average Booking Time	< 3 minutes

5. Software Requirement Specification (SRS)

5.1 Introduction

This Software Requirement Specification (SRS) document describes the functional and non-functional requirements for the GharSeva AI-Powered Home Services Marketplace. It follows IEEE 830 SRS standards and serves as the primary reference for the development, testing, and deployment teams.

5.2 Overall Description

5.2.1 Product Perspective

GharSeva is a standalone web application with mobile-responsive design. It interacts with third-party services including a payment gateway, SMS/email notification service, and mapping API (Google Maps). It exposes a RESTful API consumed by the frontend and potentially by future mobile clients.

5.2.2 Product Functions

- User registration, authentication, and profile management.
- Service listing, search, filtering, and booking.
- AI-powered provider matching and recommendation.
- Real-time booking status tracking.
- Secure payment processing.
- Ratings, reviews, and dispute resolution.
- Admin dashboard for platform management.
- Notifications via email and SMS.

5.2.3 User Classes and Characteristics

User Class	Characteristics	Technical Proficiency
Customer	Homeowner seeking services, 25–55 age group	Basic to Moderate
Service Provider	Skilled professional, may be less tech-savvy	Basic
Admin	Platform operator, manages users and content	High
Support Agent	Handles disputes and customer queries	Moderate

5.3 External Interface Requirements

5.3.1 User Interfaces

- Responsive web application built with React.js and Tailwind CSS.
- Consistent design system with accessible color contrast ratios (WCAG 2.1 AA).
- Support for modern browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+.

5.3.2 Software Interfaces

Interface	Purpose	Protocol
Razorpay API	Payment processing	HTTPS/REST
Twilio / MSG91	SMS notifications	HTTPS/REST
SendGrid	Email notifications	HTTPS/REST
Google Maps API	Location services and geocoding	HTTPS/REST

Interface	Purpose	Protocol
AWS S3 / Cloudinary	Media and document storage	HTTPS/REST
Background Verification API	Provider KYC verification	HTTPS/REST

6. Stakeholder Analysis

A comprehensive stakeholder analysis ensures that all parties with interest in or influence over the GharSeva platform are identified, their expectations understood, and their engagement strategy defined.

Stakeholder	Role	Interest	Influence	Engagement Strategy
Customers	End users booking services	Reliable, affordable, convenient service	High	User research, beta testing, feedback loops
Service Providers	Professionals delivering services	Consistent income, fair commission, tools	High	Onboarding training, support portal
Platform Admin	Operations team	Platform health, revenue, compliance	Very High	Real-time dashboards, alerts
Investors/Sponsors	Project funding	ROI, growth metrics, market share	High	Periodic reports, KPI dashboards
MCA Faculty	Academic evaluators	Technical rigor, documentation quality	Medium	Full documentation, demo sessions
Payment Gateway	Razorpay integration	Successful transactions, compliance	Medium	API integration, sandbox testing
Regulatory Bodies	Government/labor laws	Gig worker rights, tax compliance	Medium	Legal review, policy compliance docs
Support Team	Dispute resolution	Resolution SLAs, customer satisfaction	Low-Medium	Ticketing system, escalation matrix

7. Functional Requirements

7.1 Authentication & User Management

FR-ID	Requirement	Priority
FR-01	The system shall allow new users to register using email and password.	Must
FR-02	The system shall support OAuth 2.0 social login (Google).	Should
FR-03	The system shall issue JWT access tokens and refresh tokens upon successful login.	Must
FR-04	The system shall invalidate tokens upon logout.	Must
FR-05	The system shall enforce email verification before account activation.	Must
FR-06	The system shall support password reset via OTP sent to registered email.	Must
FR-07	The system shall allow users to update profile information, photo, and address.	Must
FR-08	The system shall support role-based access (Customer, Provider, Admin, Support).	Must

7.2 Service Browsing & Search

FR-ID	Requirement	Priority
FR-09	The system shall display a categorized list of all available home services.	Must
FR-10	The system shall allow full-text search with keyword suggestions.	Must
FR-11	The system shall support filtering by service type, price range, availability, and rating.	Must
FR-12	The system shall display service provider profiles including ratings, experience, and portfolio.	Must
FR-13	The system shall provide AI-powered recommendations based on user history and preferences.	Should
FR-14	The system shall display real-time provider availability on a calendar view.	Must

7.3 Booking Management

FR-ID	Requirement	Priority
FR-15	The system shall allow customers to create, view, modify, and cancel service bookings.	Must
FR-16	The system shall notify the provider upon new booking request creation.	Must
FR-17	The system shall allow providers to accept or reject booking requests.	Must
FR-18	The system shall track booking status through defined states: Pending, Confirmed, In-Progress, Completed, Cancelled.	Must

FR-ID	Requirement	Priority
FR-19	The system shall support scheduling for same-day, next-day, or future bookings.	Must
FR-20	The system shall send automated reminders 24 hours and 1 hour before the service.	Should
FR-21	The system shall calculate and display a transparent cost estimate before booking confirmation.	Must

7.4 Payment

FR-ID	Requirement	Priority
FR-22	The system shall process payments through Razorpay gateway (cards, UPI, netbanking).	Must
FR-23	The system shall generate and store digital invoices for each completed transaction.	Must
FR-24	The system shall support partial advance payments and post-service balance settlement.	Should
FR-25	The system shall process refunds within 5–7 business days for cancellations per policy.	Must
FR-26	The system shall maintain a wallet for providers to track earnings and request withdrawals.	Should

7.5 Ratings & Reviews

FR-ID	Requirement	Priority
FR-27	The system shall allow customers to rate and review providers after service completion.	Must
FR-28	The system shall calculate and display a rolling average rating for each provider.	Must
FR-29	The system shall allow providers to respond to reviews.	Should
FR-30	The system shall allow admins to flag and remove abusive reviews.	Must

7.6 Admin Dashboard

FR-ID	Requirement	Priority
FR-31	The system shall provide admins with a dashboard showing platform-wide KPIs.	Must
FR-32	The system shall allow admins to approve, suspend, or ban user accounts.	Must
FR-33	The system shall allow admins to manage service categories and pricing rules.	Must

FR-ID	Requirement	Priority
FR-34	The system shall provide reporting tools for bookings, revenue, and provider performance.	Must
FR-35	The system shall allow admins to resolve disputes and issue manual refunds.	Must

8. Non-Functional Requirements

NFR-ID	Category	Requirement	Metric
NFR-01	Performance	API response time for standard queries	< 200ms (p95)
NFR-02	Performance	Page load time (First Contentful Paint)	< 2 seconds on 4G
NFR-03	Scalability	Concurrent active users supported	> 5,000 concurrent
NFR-04	Scalability	Horizontal scaling of API tier	Via containerization
NFR-05	Availability	Platform uptime SLA	99.9% (< 8.7 hrs/year downtime)
NFR-06	Security	Data in transit encryption	TLS 1.3 mandatory
NFR-07	Security	Data at rest encryption	AES-256 for PII fields
NFR-08	Security	Authentication token expiry	Access: 15 min; Refresh: 7 days
NFR-09	Reliability	Database backup frequency	Daily full + hourly incremental
NFR-10	Reliability	Recovery Time Objective (RTO)	< 4 hours
NFR-11	Reliability	Recovery Point Objective (RPO)	< 1 hour
NFR-12	Maintainability	Code test coverage	> 80% unit test coverage
NFR-13	Usability	WCAG accessibility compliance	Level AA
NFR-14	Compliance	Data privacy standard	GDPR-aligned practices
NFR-15	Portability	Browser compatibility	Chrome, Firefox, Safari, Edge (latest 2 versions)

9. User Roles and Permissions

9.1 Role Hierarchy

GharSeva implements a four-tier role hierarchy with granular permission control. Roles are embedded in the JWT payload and validated on every protected API endpoint.

Permission	Customer	Provider	Support Agent	Admin
Register / Login	✔	✔	✔	✔
Browse Services	✔	✔	✔	✔
Create Booking	✔	✗	✗	✔
Accept/Reject Booking	✗	✔	✗	✔
Process Payments	✔	✗	✗	✔
View Own Profile	✔	✔	✔	✔
Edit Own Profile	✔	✔	✔	✔
Submit Reviews	✔ (post-service)	✗	✗	✗
Respond to Reviews	✗	✔	✗	✔
Manage Service Listings	✗	✔ (own)	✗	✔
Access Admin Dashboard	✗	✗	Partial	✔
Manage User Accounts	✗	✗	✗	✔
Generate Reports	✗	✗	✗	✔
Resolve Disputes	✗	✗	✔	✔
Issue Refunds	✗	✗	✗	✔

10. User Stories

10.1 Customer Stories

Story ID	As a...	I want to...	So that...
US-C01	Customer	Register and verify my email	I can securely access the platform
US-C02	Customer	Search for plumbers near my location	I can find available help quickly
US-C03	Customer	View a provider's profile, ratings, and reviews	I can make an informed booking decision
US-C04	Customer	Book a service for a specific date and time slot	I can plan around my schedule
US-C05	Customer	Pay securely using UPI or credit card	I can complete the transaction without risk
US-C06	Customer	Track the real-time status of my booking	I know when the provider will arrive
US-C07	Customer	Rate and review the service provider	I can share feedback to help others
US-C08	Customer	Cancel a booking and receive a refund	I am not penalized for last-minute changes

10.2 Service Provider Stories

Story ID	As a...	I want to...	So that...
US-P01	Provider	Register and upload my credentials	I can be verified and onboarded to the platform
US-P02	Provider	Set my availability calendar	Customers can book me at convenient times
US-P03	Provider	Receive notifications for new booking requests	I can respond promptly to opportunities
US-P04	Provider	View my earnings dashboard	I can track income and request withdrawals
US-P05	Provider	Respond to customer reviews	I can address feedback professionally
US-P06	Provider	Mark a job as completed	The system updates status and releases payment

10.3 Admin Stories

Story ID	As a...	I want to...	So that...
US-A01	Admin	View platform-wide booking and revenue metrics	I can monitor platform health in real time

Story ID	As a...	I want to...	So that...
US-A02	Admin	Approve or reject provider onboarding applications	Only verified professionals join the platform
US-A03	Admin	Suspend abusive users or providers	Platform integrity is maintained
US-A04	Admin	Manually issue a refund for a disputed transaction	Customer trust is preserved
US-A05	Admin	Generate weekly performance reports	I can make data-driven business decisions

11. Use Cases

11.1 Use Case: Book a Home Service

Field	Detail
Use Case ID	UC-001
Use Case Name	Book a Home Service
Actor	Customer
Precondition	Customer is registered, email-verified, and logged in.
Trigger	Customer selects a service category and initiates a booking.
Main Flow	1. Customer searches for a service. 2. System displays matching providers. 3. Customer selects a provider and views profile. 4. Customer selects date, time, and address. 5. System displays cost estimate. 6. Customer confirms booking and proceeds to payment. 7. Payment is processed via Razorpay. 8. System creates booking record with status 'Pending'. 9. Provider receives notification. 10. Provider accepts booking; status changes to 'Confirmed'. 11. Customer and provider receive confirmation with details.
Alternative Flow	AF1: Provider rejects – System notifies customer and offers re-matching. AF2: Payment fails – Booking is not created; customer is notified to retry.
Postcondition	Booking is confirmed; payment is captured; notifications sent.
Exception	If no providers are available, system suggests alternative dates.

11.2 Use Case: Provider Onboarding

Field	Detail
Use Case ID	UC-002
Use Case Name	Service Provider Onboarding
Actor	Service Provider, Admin
Precondition	Provider has a valid government-issued ID and service skill certifications.
Trigger	Provider submits onboarding application via the registration form.
Main Flow	1. Provider registers with email and password. 2. Provider fills profile: name, skills, service areas, experience. 3. Provider uploads ID proof and skill certificates. 4. System stores application in 'Under Review' status. 5. Admin reviews the application and documents. 6. Admin approves; provider receives activation email. 7. Provider sets availability and starts accepting bookings.
Alternative Flow	Admin rejects with reason; provider can re-apply after corrections.
Postcondition	Provider account is active and discoverable in the marketplace.

11.3 Use Case: AI-Powered Provider Matching

Field	Detail
Use Case ID	UC-003
Use Case Name	AI Provider Matching
Actor	System (Automated), Customer
Precondition	Customer has specified service type, location, and preferred date/time.
Trigger	Customer initiates a service search or requests auto-match.
Main Flow	1. System receives search parameters. 2. AI module filters providers by: availability, service category, geographic proximity (< 10 km), and minimum rating threshold (> 3.5). 3. Providers are scored using a weighted algorithm: Rating (30%), Distance (25%), Completion Rate (25%), Response Time (20%). 4. Top 5 matches are returned, ranked by score. 5. Customer selects preferred provider.
Postcondition	Customer is presented with ranked provider recommendations.

12. System Scope

12.1 In Scope

The following capabilities are included in the GharSeva MVP:

- Web-based platform accessible via desktop and mobile browsers.
- Eight home service categories as defined in Section 4.3.
- Customer and provider registration with email verification.
- AI-powered search and provider matching engine.
- Booking lifecycle management (create, track, complete, cancel).
- Secure payment integration with Razorpay.
- Post-service ratings and reviews.
- Admin dashboard for user management and reporting.
- Notification system (email + SMS).
- Responsive UI supporting mobile and desktop.

12.2 Out of Scope

The following are explicitly excluded from the current release:

- Native iOS and Android mobile applications.
- Live video consultation feature.
- Multi-language support (Phase 2 roadmap).
- Subscription-based service plans for customers.
- AI-based automated pricing engine.
- Integration with IoT smart home devices.
- In-app messaging and real-time chat between customer and provider.

13. Assumptions and Constraints

13.1 Assumptions

ID	Assumption
A-01	Users have access to internet connectivity with minimum 2G speed.
A-02	Service providers possess a smartphone or access to a web browser.
A-03	Payment gateway Razorpay is available and operational in all target geographies.
A-04	Google Maps API will provide accurate geocoding for Indian pin codes.
A-05	Third-party background verification API will respond within 48 hours for KYC checks.
A-06	Email and SMS delivery rates will exceed 95% in target markets.
A-07	All monetary transactions are in Indian Rupees (INR) for the MVP.
A-08	PostgreSQL database version 14+ is available in the production environment.

13.2 Constraints

ID	Constraint
C-01	The project must be completed within the MCA academic timeline (6 months).
C-02	Frontend must support browsers released within the last 2 major versions.
C-03	The system must comply with the Information Technology Act, 2000 (India).
C-04	API rate limits imposed by third-party services (Google Maps: 1000 req/day on free tier).
C-05	The initial deployment budget is constrained to available cloud free-tier resources.
C-06	The MVP will only support English language content.
C-07	JWT secret keys must be rotated every 90 days per security policy.
C-08	File uploads are limited to 5MB per file for provider documents.

14. System Architecture Description

14.1 Architectural Overview

GharSeva follows a layered, microservice-ready monolithic architecture for the MVP phase, with clear separation of concerns across presentation, business logic, and data layers. The architecture is designed for progressive decomposition into microservices as the platform scales.

14.2 Architecture Layers

Layer	Technology	Responsibility
Presentation Layer	React.js + Tailwind CSS	Renders UI, handles user interactions, manages client-side state via React Context/Redux
API Gateway Layer	FastAPI (Python 3.11+)	Exposes RESTful API endpoints, handles routing, authentication middleware, and rate limiting
Business Logic Layer	FastAPI Services + AI Module	Implements booking workflows, AI matching logic, payment orchestration, and notification triggers
Data Access Layer	SQLAlchemy ORM	Abstracts database queries; implements repository pattern for clean data access
Database Layer	PostgreSQL 14+	Stores all persistent data including users, bookings, transactions, and reviews
Cache Layer	Redis (Phase 2)	Session cache, API response caching, rate limit counters
Storage Layer	AWS S3 / Cloudinary	Stores user profile photos, provider documents, and service images
Notification Layer	Celery + Redis + SendGrid/Twilio	Asynchronous task queue for email and SMS notifications

14.3 Component Interaction Flow

The request lifecycle follows this path: Browser → HTTPS → Nginx Reverse Proxy → FastAPI Application Server → SQLAlchemy ORM → PostgreSQL. Asynchronous operations (notifications, AI matching at scale) are offloaded to Celery workers. Static assets are served via CDN (CloudFront).

14.4 Key Architectural Decisions

Decision	Rationale
FastAPI over Django REST	FastAPI offers native async support, automatic OpenAPI docs, and better performance for I/O-bound workloads
PostgreSQL over MySQL	Superior JSON support, JSONB indexing for flexible fields, full-text search, and better concurrency

Decision	Rationale
JWT Stateless Auth	Eliminates server-side session storage, enabling horizontal scaling without sticky sessions
SQLAlchemy ORM	Provides Pythonic database access, supports migrations via Alembic, and abstracts vendor-specific SQL
React.js + Tailwind CSS	Component reusability, fast rendering via virtual DOM, utility-first CSS for consistent design
Monolith for MVP	Reduces operational complexity; architecture supports future decomposition into microservices

15. Database Design

15.1 Core Entities and Tables

15.1.1 Users Table

Column	Data Type	Constraints	Description
user_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED	User email address
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
full_name	VARCHAR(150)	NOT NULL	User's full name
phone	VARCHAR(15)	UNIQUE	Mobile number for SMS
role	ENUM	NOT NULL (customer/provider/admin/support)	User role
is_verified	BOOLEAN	DEFAULT FALSE	Email verification status
is_active	BOOLEAN	DEFAULT TRUE	Account active status
profile_photo_url	TEXT	NULLABLE	URL to profile image
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time
updated_at	TIMESTAMP	ON UPDATE NOW()	Last modification time

15.1.2 Service Providers Table

Column	Data Type	Constraints	Description
provider_id	UUID	PRIMARY KEY	Provider identifier
user_id	UUID	FK → users(user_id)	Links to user account
bio	TEXT	NULLABLE	Provider description
experience_years	INTEGER	DEFAULT 0	Years of experience
avg_rating	DECIMAL(3,2)	DEFAULT 0.00	Computed average rating
total_jobs	INTEGER	DEFAULT 0	Total completed jobs
is_available	BOOLEAN	DEFAULT TRUE	Current availability flag
service_radius_km	INTEGER	DEFAULT 10	Operating radius in km
verification_status	ENUM	pending/approved/rejected	KYC verification status
bank_account_info	JSONB	ENCRYPTED	Encrypted payment details

15.1.3 Bookings Table

Column	Data Type	Constraints	Description
booking_id	UUID	PRIMARY KEY	Unique booking reference
customer_id	UUID	FK → users(user_id)	Booking customer
provider_id	UUID	FK → service_providers(provider_id)	Assigned provider
service_id	UUID	FK → services(service_id)	Booked service
status	ENUM	pending/confirmed/in_progress/completed/cancelled	Booking state
scheduled_at	TIMESTAMP	NOT NULL	Scheduled service date/time
address_id	UUID	FK → addresses(address_id)	Service location
total_amount	DECIMAL(10,2)	NOT NULL	Total booking cost
notes	TEXT	NULLABLE	Customer special instructions
created_at	TIMESTAMP	DEFAULT NOW()	Booking creation timestamp

15.2 Additional Tables Summary

Table Name	Purpose	Key Relationships
services	Master list of service types and pricing	Referenced by bookings, provider_services
service_categories	Top-level service groupings (e.g., Plumbing)	Parent of services table
provider_services	Many-to-many: providers ↔ services	Links providers to offered services
payments	Records all payment transactions	One-to-one with bookings
reviews	Customer reviews and ratings post-service	FK to bookings, customers, providers
addresses	Customer and provider location data	Referenced by users, bookings
notifications	Log of all sent notifications	FK to users
disputes	Dispute tickets raised by users	FK to bookings, users
provider_availability	Provider schedule/calendar blocks	FK to providers
otp_tokens	OTP storage for email/phone verification	FK to users, TTL-indexed

16. Entity Relationship Diagram Description

16.1 Entity Relationships

The following describes the logical entity relationships within the GharSeva database schema:

Relationship	Type	Description
User → ServiceProvider	One-to-One	A user with the 'provider' role has exactly one provider profile.
User → Booking (as Customer)	One-to-Many	A customer can create multiple bookings over time.
ServiceProvider → Booking	One-to-Many	A provider can fulfill multiple bookings.
Booking → Payment	One-to-One	Each booking has exactly one associated payment record.
Booking → Review	One-to-One	A review can be submitted once per completed booking.
ServiceProvider ↔ Services	Many-to-Many	Resolved via provider_services junction table.
ServiceCategory → Services	One-to-Many	Each category contains multiple services.
User → Address	One-to-Many	A user can save multiple service addresses.
Booking → Address	Many-to-One	Multiple bookings can reference the same address.
User → Notification	One-to-Many	Each user has a log of notifications received.
Booking → Dispute	One-to-One	A dispute can be raised per problematic booking.

16.2 Key Constraints and Indexes

- Composite index on bookings (customer_id, status) for fast status-based lookups.
- Partial index on users (email) WHERE is_verified = TRUE for authenticated queries.
- GiST spatial index on addresses (coordinates) for geographic proximity queries.
- Composite unique constraint on reviews (booking_id, customer_id) to prevent duplicate reviews.
- CHECK constraint on bookings: scheduled_at > created_at to prevent past bookings.
- Foreign key constraints with ON DELETE RESTRICT on all user-linked tables to prevent orphaned records.

17. API Specification

17.1 API Design Principles

- RESTful design following OpenAPI 3.0 specification.
- All endpoints prefixed with /api/v1/.
- JSON request and response bodies throughout.
- HTTP status codes used semantically (200, 201, 400, 401, 403, 404, 422, 500).
- JWT Bearer token required for all protected endpoints.

17.2 Authentication Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/v1/auth/register	Register new user account	No
POST	/api/v1/auth/login	Authenticate and receive JWT tokens	No
POST	/api/v1/auth/logout	Invalidate refresh token	Yes
POST	/api/v1/auth/refresh	Obtain new access token	Refresh Token
POST	/api/v1/auth/verify-email	Verify email with OTP	No
POST	/api/v1/auth/forgot-password	Request password reset OTP	No
POST	/api/v1/auth/reset-password	Set new password using OTP	No

17.3 Booking Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/v1/bookings	List all bookings for authenticated user	Yes
POST	/api/v1/bookings	Create new booking request	Yes (Customer)
GET	/api/v1/bookings/{id}	Get single booking details	Yes
PATCH	/api/v1/bookings/{id}/status	Update booking status	Yes (Provider/Admin)
DELETE	/api/v1/bookings/{id}	Cancel a booking	Yes (Customer/Admin)
GET	/api/v1/bookings/{id}/invoice	Download invoice PDF	Yes

17.4 Provider Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/v1/providers	Search and list providers with filters	No
GET	/api/v1/providers/{id}	Get provider public profile	No

Method	Endpoint	Description	Auth Required
PUT	/api/v1/providers/{id}	Update provider profile	Yes (Provider/Admin)
GET	/api/v1/providers/{id}/availability	Get provider calendar availability	No
POST	/api/v1/providers/{id}/availability	Set availability slots	Yes (Provider)
GET	/api/v1/providers/match	AI-powered provider matching	Yes (Customer)

17.5 Payment Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/v1/payments/initiate	Initiate Razorpay payment order	Yes
POST	/api/v1/payments/verify	Verify payment signature from gateway	Yes
POST	/api/v1/payments/refund	Issue refund for cancelled booking	Yes (Admin)
GET	/api/v1/payments/history	Get payment history for user	Yes

18. UI/UX Screen List

Screen ID	Screen Name	User Role	Key Features
UI-01	Landing Page	Public	Hero section, service categories, testimonials, CTA buttons
UI-02	Registration Page	Public	Email/password form, social login, terms acceptance
UI-03	Login Page	Public	Email/password login, forgot password link, social login
UI-04	Email Verification	Public	OTP input, resend option, success confirmation
UI-05	Customer Home Dashboard	Customer	Quick search, recommended services, recent bookings, notifications
UI-06	Service Category Browse	Customer	Category cards, sub-category filters, service list grid
UI-07	Provider Search Results	Customer	Provider cards with rating, price, distance; filter panel; map view
UI-08	Provider Profile Page	Customer	Photo, bio, services, ratings breakdown, availability calendar
UI-09	Booking Flow – Step 1	Customer	Date/time picker, slot availability, address input
UI-10	Booking Flow – Step 2	Customer	Cost breakdown, special instructions, booking summary
UI-11	Payment Page	Customer	Razorpay widget, order summary, secure payment badge
UI-12	Booking Confirmation	Customer	Booking ID, provider details, scheduled time, action buttons
UI-13	My Bookings List	Customer	Tabbed view: Upcoming, Active, Completed, Cancelled
UI-14	Booking Detail Page	Customer/Provider	Full booking info, status timeline, contact info, actions
UI-15	Review Submission Form	Customer	Star rating, written review, photo upload option
UI-16	Customer Profile	Customer	Personal info, saved addresses, notification preferences
UI-17	Provider Dashboard	Provider	Earnings summary, upcoming jobs, acceptance rate, reviews
UI-18	Provider Job Management	Provider	Incoming requests, active jobs, job history
UI-19	Provider Profile Edit	Provider	Bio, skills, certifications, photo, service areas

Screen ID	Screen Name	User Role	Key Features
UI-20	Provider Availability	Provider	Weekly calendar, time slot toggle, day-off management
UI-21	Admin Dashboard	Admin	KPIs: Users, Bookings, Revenue, Issues; charts; alerts
UI-22	User Management (Admin)	Admin	Searchable table, filters, bulk actions, account actions
UI-23	Provider Verification (Admin)	Admin	Application queue, document viewer, approve/reject action
UI-24	Dispute Management	Admin/Support	Dispute list, timeline, evidence, resolution actions
UI-25	Reports & Analytics	Admin	Date-range filters, export CSV, charts by category/region

19. Security Requirements

19.1 Authentication & Authorization

- All passwords must be hashed using bcrypt with a minimum cost factor of 12.
- JWT access tokens expire in 15 minutes; refresh tokens expire in 7 days.
- Role-based access control (RBAC) enforced at the FastAPI dependency level.
- Account lockout after 5 consecutive failed login attempts; cooldown of 30 minutes.
- Multi-factor authentication (MFA) available for admin accounts.

19.2 Data Security

- All API communication over HTTPS/TLS 1.3; HTTP redirects to HTTPS with HSTS.
- Sensitive PII fields (phone, bank info) encrypted at rest using AES-256.
- PostgreSQL connection uses SSL; connection string stored in environment variables, never in code.
- AWS S3 buckets are private; file access is via pre-signed URLs with TTL of 10 minutes.

19.3 API Security

Security Control	Implementation
Input Validation	Pydantic models enforce strict type validation on all API inputs
SQL Injection Prevention	SQLAlchemy ORM with parameterized queries; raw SQL prohibited
XSS Prevention	React auto-escapes JSX output; Content Security Policy headers enforced
CSRF Protection	SameSite cookie attribute; CSRF token for state-changing operations
Rate Limiting	FastAPI middleware limits to 100 req/min per IP; stricter on auth endpoints
CORS Policy	Strict CORS whitelist of production domain only
Secrets Management	Secrets stored in AWS Parameter Store / environment variables
Dependency Scanning	Snyk / Dependabot automated vulnerability scanning in CI pipeline

19.4 Compliance

- GDPR-aligned data practices: right to erasure, data portability, and explicit consent at registration.
- Audit log maintained for all admin actions with user ID, action, timestamp, and IP.
- Payment card data is never stored on platform; Razorpay handles PCI-DSS compliance.

20. Testing Strategy

20.1 Testing Levels

Testing Level	Scope	Tools	Target Coverage
Unit Testing	Individual functions, service methods, utility classes	pytest (backend), Jest + React Testing Library (frontend)	> 80%
Integration Testing	API endpoint to database interaction, service layer flows	pytest + TestClient (FastAPI), test PostgreSQL DB	> 70%
System Testing	End-to-end user flows: registration, booking, payment	Playwright / Cypress	All critical paths
Performance Testing	API throughput, response time under concurrent load	Locust / k6	5,000 concurrent users
Security Testing	OWASP Top 10 vulnerability assessment, pen testing	OWASP ZAP, Bandit (Python)	Zero critical findings
User Acceptance Testing	Stakeholder validation of business requirements	Manual, with test scripts	100% BRD items
Regression Testing	Verify new changes don't break existing functionality	Automated CI pipeline	On every PR merge

20.2 Test Environments

Environment	Purpose	Data
Local Dev	Developer unit and integration tests	Mocked / seeded test data
Staging	UAT and end-to-end testing	Anonymized production-like data
Production	Live monitoring, smoke tests post-deployment	Real data

20.3 Defect Management

Defects are tracked using GitHub Issues with the following severity taxonomy:

Severity	Definition	Response SLA
Critical (P0)	Platform down, data loss, security breach	Fix within 4 hours
High (P1)	Core feature broken, payment failure, data integrity issue	Fix within 24 hours
Medium (P2)	Feature partially broken, workaround exists	Fix within 3 business days
Low (P3)	Minor UI issue, cosmetic defect, minor UX friction	Fix in next sprint

21. Deployment Strategy

21.1 Deployment Architecture

GharSeva uses a containerized deployment model with Docker and Docker Compose for local and staging environments, progressing to AWS ECS (Elastic Container Service) for production with an RDS PostgreSQL managed database instance.

21.2 Environment Configuration

Environment	Infrastructure	Deployment Method
Development	Local machine / Docker Compose	Developer manually runs docker-compose up
Staging	AWS EC2 (t3.medium) + RDS (db.t3.micro)	GitHub Actions CI/CD on merge to 'develop' branch
Production	AWS ECS Fargate + RDS PostgreSQL (Multi-AZ)	GitHub Actions CD on merge to 'main' + manual approval gate

21.3 CI/CD Pipeline Stages

Stage	Action	Trigger
1. Code Commit	Developer pushes feature branch	Manual
2. Lint & Format	ESLint (frontend), Black + flake8 (backend)	On every push
3. Unit Tests	Run full unit test suite with coverage check	On every push
4. Build Docker Images	Build frontend (Nginx) and backend (Uvicorn) images	On PR creation
5. Integration Tests	Run API integration tests against staging DB	On PR to develop
6. Security Scan	OWASP ZAP, Bandit, Snyk dependency scan	On PR to develop
7. Deploy to Staging	Push images to ECR, update ECS task definition	On merge to develop
8. Smoke Tests	Automated health checks and critical path tests	Post-staging deploy
9. Deploy to Production	Same as staging with manual approval gate	On merge to main

21.4 Rollback Strategy

- Blue-Green deployment strategy ensures zero downtime during releases.
- Previous ECS task definition version is retained for immediate rollback.
- Database migrations use Alembic with reversible down migrations.
- Rollback can be triggered within 5 minutes via AWS Console or CLI.

22. Future Scope

Phase	Feature	Description	Timeline
Phase 2	Native Mobile Apps	iOS and Android apps using React Native for push notifications and GPS tracking	Q3 2026
Phase 2	In-App Chat	Real-time messaging between customer and provider via WebSocket	Q3 2026
Phase 2	Multi-Language Support	Hindi, Tamil, Telugu, and Marathi language support	Q4 2026
Phase 3	AI Demand Forecasting	ML model predicting service demand by geography and season for provider recommendations	Q1 2027
Phase 3	Subscription Plans	Monthly/annual plans for customers offering discounted bookings and priority matching	Q1 2027
Phase 3	Dynamic Pricing Engine	AI-driven surge pricing during peak demand periods	Q2 2027
Phase 4	IoT Integration	Smart home integration for automated service triggers (e.g., AC service reminder)	Q3 2027
Phase 4	AR Service Preview	Augmented reality for customers to visualize service outcomes (e.g., painting colors)	Q4 2027
Phase 4	B2B Corporate Services	Bulk service contracts for offices, co-working spaces, and residential societies	Q4 2027

23. Success Metrics

23.1 Business Metrics

Metric	Definition	6-Month Target	12-Month Target
Monthly Active Users	Unique users with at least one session per month	10,000	50,000
Bookings Per Month	Total confirmed service bookings	5,000	25,000
Gross Merchandise Value (GMV)	Total transaction value processed	INR 25 Lakhs	INR 1.5 Crores
Customer Retention Rate	% customers who book again within 90 days	35%	55%
Provider Utilization Rate	% of available slots that are booked	40%	65%
Net Promoter Score (NPS)	Customer satisfaction and advocacy measure	> 30	> 50

23.2 Technical Metrics

Metric	Target
API Uptime	99.9% (measured monthly)
Mean Time to Recovery (MTTR)	< 30 minutes for P0 incidents
Average API Response Time	< 200ms at 95th percentile
Deployment Frequency	Minimum 2 releases per week
Change Failure Rate	< 5% of deployments require rollback
Security Vulnerability Remediation	Critical: within 4 hours; High: within 24 hours

24. Conclusion

GharSeva represents a comprehensive, technology-forward solution to the deeply fragmented and underserved home services market in India. By combining the power of artificial intelligence with a robust, scalable technical architecture, the platform addresses real consumer pain points while creating economic opportunity for skilled service professionals.

This Software Project Documentation captures the complete lifecycle requirements of the GharSeva platform — from business vision and stakeholder alignment through to technical specifications, security controls, testing strategy, and deployment planning. Every design decision reflects industry best practices and is grounded in the realities of building production-grade software at scale.

The chosen technology stack — React.js with Tailwind CSS, FastAPI, PostgreSQL, SQLAlchemy, and JWT — provides an optimal balance of developer productivity, runtime performance, and maintainability. The architecture is deliberately designed to support the MVP phase while providing a clear path for evolution into a distributed microservices system as the business scales.

From an academic perspective, this project demonstrates mastery of full-stack software engineering principles, database design, API development, system architecture, and modern software development lifecycle practices — core competencies of the Master of Computer Applications program.

It is anticipated that GharSeva, upon successful completion and deployment, will serve as a benchmark for technology-driven disruption in the home services sector, delivering measurable value to customers, providers, and stakeholders alike.

— End of Document —

GharSeva – AI-Powered Home Services Marketplace

MCA Project Documentation v1.0 | July 2026